

Projektaufgabe

Erstes eigenes Projekt – Datenanwendung mit Python

Lernziele

Nach Abschluss dieses Projekts können Sie:

- eine eigene Anwendung von der Anforderungsanalyse bis zur Übergabe selbstständig planen und umsetzen,
- ein Frontend, ein Backend und eine persistente Datenquelle zu einem funktionierenden Gesamtsystem verbinden,
- externe Daten über eine REST-API beziehen, verarbeiten und nutzergerecht darstellen,
- Datenanalyse mit **pandas** oder vergleichbaren Bibliotheken praktisch einsetzen,
- eine saubere Projektstruktur aufbauen und Ihre Designentscheidungen begründen,
- Aspekte des Datenschutzes und der Datensicherheit konkret in Ihrem Projekt berücksichtigen,
- KI-Werkzeuge bewusst und reflektiert einsetzen,
- Ihre Lösung verständlich präsentieren, dokumentieren und kritisch reflektieren.

Projektüberblick

Willkommen zu Ihrem ersten eigenen Anwendungsprojekt. Bis hierhin haben Sie in vielen kleinen Übungen einzelne Bausteine kennengelernt – jetzt setzen Sie diese zum ersten Mal zu einer richtigen Anwendung zusammen. Das ist ein wichtiger Schritt: Sie werden merken, dass das Zusammenspiel der Teile oft anspruchsvoller ist als jeder einzelne Teil für sich.

Sie haben **sieben Arbeitstage** Zeit, um Ihre Anwendung zu entwerfen, zu bauen, zu dokumentieren und vorzustellen. Davor steht ein **Vorbereitungstag**, an dem die wichtigsten Werkzeuge, Bibliotheken und Vorgehensweisen vorgestellt werden. Die tiefere Einarbeitung in die einzelnen Themen erfolgt während des Projekts – selbstständig und nach Bedarf. Das ist Absicht: Sich in neue Themen einzuarbeiten zu können gehört zum Berufsalltag jedes Fachinformatikers.

Wichtig!

Ziel ist nicht die perfekte Anwendung. Ziel ist, dass Sie einen vollständigen Entwicklungsprozess durchlaufen – von der ersten Idee über die Anforderungsanalyse bis zur Übergabe und Präsentation. Wenn Sie am Ende ein laufendes, gut dokumentiertes Programm haben, das Sie selbst verstehen und

erklären können, war das Projekt erfolgreich.

Worum geht es?

Sie entwickeln eine **Datenanwendung**. Damit ist ein Programm gemeint, das Daten aus verschiedenen Quellen zusammenführt, sie verarbeitet und einer Nutzerin oder einem Nutzer in einer brauchbaren Form zur Verfügung stellt. Solche Anwendungen sind im Berufsalltag häufig – ob im Controlling, im technischen Service, in der Logistik oder im Bereich Business Intelligence.

Sie haben die Wahl zwischen zwei Themen, die beide diese Anforderungen erfüllen:

1. **Variante A – Strompreis- und Energieassistent:** Eine Anwendung, die aktuelle Börsenstrompreise abrufen, sie mit Verbrauchsdaten zusammenführt und Empfehlungen für günstige Verbrauchszeiten gibt.
2. **Variante B – Reise- und Städte-Dashboard:** Eine Anwendung, die Wetter- und Länderdaten für selbst gewählte Reiseziele bündelt, vergleicht und für die Reiseplanung aufbereitet.

Beide Varianten haben denselben Pflichtumfang und werden gleich bewertet. Sie können die Variante frei wählen – entscheiden Sie sich für das Thema, das Sie persönlich mehr interessiert. Details zu beiden Varianten finden Sie in Abschnitt .

Was Sie mitbringen

Sie haben in den vergangenen Wochen die Grundlagen der Programmierung in Python und die Grundzüge der Softwareentwicklung kennengelernt. Sie wissen, was Variablen, Funktionen, Schleifen, Listen und Wörterbücher sind. Sie haben einfache Programme geschrieben, mit Dateien gearbeitet und erste Erfahrungen mit Modularisierung gesammelt.

Was Sie sich im Projekt **neu erarbeiten** werden – das ist Teil der Aufgabe:

- das Arbeiten mit einer Datenbank oder dem strukturierten Lesen und Schreiben von Excel,
- das Aufrufen einer externen API über HTTP,
- die Datenverarbeitung mit **pandas**,
- den Aufbau eines einfachen Frontends (z. B. mit Streamlit oder Flask),
- das Strukturieren eines Projekts mit mehreren Modulen.

Am Vorbereitungstag bekommen Sie zu jedem dieser Themen einen Überblick und Hinweise, wo Sie weiterlesen können. Die eigentliche Einarbeitung findet während des Projekts statt.

Rahmenbedingungen auf einen Blick

Definition

Bearbeitungszeit: 7 Tage (Tag 1 bis Tag 7)

Vorbereitungstag: Tag 0 – gemeinsamer Überblickstag

Arbeitsform: Einzelarbeit

Code-Review: Auf freiwilliger Basis paarweise möglich (siehe Abschnitt)

Programmiersprache: Python (mindestens Version 3.10)

Abgabe: ZIP-Archiv mit Quellcode, Dokumentation, Anforderungsanalyse

Präsentation: 8–10 Minuten pro Person am letzten Tag

Pflichtanforderungen

Dieser Abschnitt beschreibt, was Ihre Anwendung am Ende mindestens leisten muss – unabhängig davon, für welche Variante Sie sich entscheiden. Lesen Sie diesen Abschnitt sorgfältig: Er ist die Grundlage für Ihre eigene Anforderungsanalyse, die Sie als erstes erstellen werden.

Anforderungsanalyse

Bevor Sie eine Zeile Code schreiben, erstellen Sie eine **Anforderungsanalyse**. Das ist ein kurzes Dokument (zwei bis vier Seiten reichen aus), in dem Sie für sich selbst und für eine fiktive Auftraggeberin festhalten:

- Was soll die Anwendung können? (Funktionale Anforderungen)
- Wer sind die Nutzerinnen und Nutzer? Was wollen diese erreichen? (User Stories)
- Welche Eigenschaften muss die Anwendung haben, jenseits der reinen Funktion? (Nicht-funktionale Anforderungen, z. B. Bedienbarkeit, Verfügbarkeit, Datenschutz)
- Welche Abgrenzungen gibt es? Was soll die Anwendung explizit *nicht* können? (Abgrenzung)
- Welche Risiken sehen Sie und wie wollen Sie ihnen begegnen?

Eine Vorlage für die Anforderungsanalyse finden Sie im Begleitmaterial (**Anforderungsanalyse-Vorlage**). Sie müssen die Vorlage nicht 1:1 nutzen – sie soll Ihnen einen Einstieg geben. Sie können die Anforderungsanalyse auch in Markdown, LaTeX oder einem anderen Format schreiben, solange das Ergebnis in Ihrer Abgabe enthalten ist.

Praxistipp

Die Anforderungsanalyse ist kein einmaliges Dokument, das Sie abhaken. Es ist normal, dass Sie nach zwei Tagen merken: ?Diese Funktion brauche ich gar

nicht? oder ?Hier habe ich etwas Wichtiges vergessen.? Pflegen Sie das Dokument während des Projekts – am Ende sollten Anforderungsanalyse und tatsächlich umgesetztes Programm zueinander passen.

Funktionale Pflichtenforderungen

Merksatz

Ihre Anwendung muss diese sechs Punkte erfüllen:

1. **Frontend:** Eine bedienbare Benutzeroberfläche (grafisch oder Web-basiert)
2. **Backend:** Eine klar abgegrenzte Programmlogik, die die Arbeit erledigt
3. **Datenquelle:** Eine persistente Datenquelle – Datenbank oder Excel-Datei
4. **API:** Mindestens eine externe REST-API wird angesprochen
5. **Datenverarbeitung:** Daten werden mit `pandas` (oder Vergleichbarem) ausgewertet
6. **Struktur und Dokumentation:** Sauber gegliederter Code mit Doku

Im Folgenden werden die sechs Punkte einzeln erläutert.

Frontend – Bedienbare Oberfläche

Ihre Anwendung muss bedient werden können, ohne dass die Nutzerin den Quellcode öffnet. Das bedeutet: Es gibt eine Oberfläche, über die Eingaben gemacht und Ergebnisse angezeigt werden. ?Oberfläche? kann sehr verschieden aussehen:

- eine grafische Oberfläche mit Eingabefeldern und Schaltflächen,
- eine Webseite, die im Browser geöffnet wird,
- ein Dashboard, das Diagramme und Tabellen darstellt.

Eine reine Konsolenanwendung (`input()` / `print()`) erfüllt diese Anforderung *nicht*. Sie dürfen Konsolen-Tools zusätzlich bauen, etwa zur Datenpflege, aber das Kernerlebnis Ihrer Anwendung muss eine richtige Oberfläche haben.

Hinweis

Empfehlung für den schnellen Einstieg: **Streamlit**. Damit bekommen Sie mit sehr wenig Code ein ansehnliches Dashboard mit Eingabefeldern, Diagrammen und Tabellen. Wer eine echte Webanwendung mit eigenen Routen und HTML-Templates möchte: **Flask**. Wer eine klassische Desktop-GUI bevorzugt: **tkinter** oder **customtkinter**. Details zu allen drei Optionen finden Sie im Cheat-Sheet und

in der Vorbereitungspräsentation.

Backend – Programmlogik trennen

Das **Backend** ist der Teil Ihrer Anwendung, der die eigentliche Arbeit macht: Daten abrufen, verarbeiten, speichern, auswerten. Wichtig ist die **Trennung** zwischen Oberfläche und Logik:

- Die Oberfläche enthält keine API-Aufrufe und keine Datenbank-Befehle.
- Die Logik weiß nichts über die konkrete Oberfläche.

Praktisch bedeutet das: Die Geschäftslogik liegt in eigenen Modulen, die von der Oberfläche aufgerufen werden. Diese Trennung macht Ihre Anwendung testbar, wartbar und austauschbar. Sie wäre theoretisch in der Lage, Ihr Frontend gegen ein anderes auszutauschen, ohne die Logik anzufassen.

Datenquelle – Persistente Speicherung

Daten, die Ihr Programm verarbeitet oder erzeugt, müssen **persistent** gespeichert werden. **Persistent** heißt: Sie bleiben erhalten, auch wenn das Programm geschlossen wird. Sie haben zwei Optionen:

1. **Datenbank:** Empfohlen wird **SQLite**. SQLite ist eine Datei-basierte Datenbank, die ohne Server-Installation funktioniert und in Python von Haus aus unterstützt wird. Wer möchte, kann auch **PostgreSQL** oder **MySQL** nutzen – das ist anspruchsvoller in der Einrichtung.
2. **Excel-Datei:** Mit der Bibliothek **openpyxl** können Sie Excel-Dateien lesen und schreiben. Für strukturierte Daten ist Excel okay, aber bei wachsenden Datenmengen oder häufigen Änderungen wird eine Datenbank schnell überlegen.

Was Sie speichern, hängt von Ihrer Variante ab – siehe Abschnitt . Die Wahl zwischen Datenbank und Excel ist eine **Designentscheidung**: Begründen Sie sie in Ihrer Anforderungsanalyse und später in der Präsentation.

Praxistipp

Auch wenn Sie mit Excel arbeiten: Wagen Sie sich mindestens einmal an SQLite heran. Datenbankverständnis ist eine Kernfertigkeit von Anwendungsentwicklern. Spätestens im Berufsalltag begegnen Sie SQL überall.

API – Externe Daten einbinden

Ihre Anwendung ruft Daten von mindestens einer **REST-API** ab. Welche API Sie nutzen, hängt von Ihrer Variante ab. Die Anbindung erfolgt typischerweise mit der Bibliothek **requests**. Sie müssen die JSON-Antwort der API verstehen, die für Sie relevanten Felder extrahieren und mit Fehlersituationen umgehen können.

Wichtig!

APIs sind im Internet erreichbare Dienste. Das heißt: Sie funktionieren nicht immer. Ihre Anwendung muss damit umgehen, wenn die API kurzzeitig nicht erreichbar ist, ungültige Daten liefert oder ein Limit überschritten wird. Wer das nicht abfängt, hat eine Anwendung, die regelmäßig mit einem unverständlichen Fehler abstürzt.

Datenverarbeitung – pandas einsetzen

Daten kommen selten in der Form an, in der sie gebraucht werden. Sie müssen sie filtern, gruppieren, sortieren, zusammenfassen oder mit anderen Daten verknüpfen. Genau dafür ist **pandas** gemacht.

Sie müssen **pandas** kein Profi werden. Aber Sie sollten mindestens zeigen, dass Sie

- Daten in einen **DataFrame** laden können,
- Spalten auswählen und filtern können,
- einfache Aggregationen durchführen (z. B. Durchschnitt, Minimum, Maximum, Anzahl),
- eine sinnvolle Auswertung erzeugen, die in Ihrer Anwendung dargestellt wird.

Wenn **pandas** für Ihre konkrete Datenmenge **überdimensioniert** wirkt: Das ist okay. Sie üben hier eine Bibliothek, die in größeren Projekten unverzichtbar ist.

Struktur und Dokumentation

Ihr Projekt besteht aus **mindestens vier separaten Modulen** plus Einstiegspunkt. Eine empfohlene Struktur finden Sie in Abschnitt . Außerdem dokumentieren Sie:

- **Docstrings** an jeder Funktion und Klasse (Was tut sie? Welche Parameter? Welcher Rückgabewert?),
- eine **README.md** mit Installation, Bedienung und kurzem Überblick,
- eine **requirements.txt** mit allen verwendeten Bibliotheken,
- eine kurze **Projektdokumentation** (1–2 Seiten), in der Sie Ihre wichtigsten Designentscheidungen begründen.

Datenschutz und Datensicherheit

Sie sind angehende Fachinformatikerinnen und Fachinformatiker. In dieser Rolle werden Sie immer wieder mit personenbezogenen oder schutzwürdigen Daten zu tun haben. Es ist daher Pflichtteil dieses Projekts, sich aktiv mit Datenschutz und Datensicherheit auseinanderzusetzen.

Definition

Mindestens diese Aspekte berücksichtigen Sie:

1. **Keine Geheimnisse im Code:** API-Schlüssel, Passwörter oder Tokens gehören nicht in den Quellcode. Verwenden Sie eine Konfigurationsdatei oder Umgebungsvariablen und schließen Sie diese aus der Versionskontrolle aus (`.gitignore`).
2. **Eingaben validieren:** Eingaben von außen – ob aus dem Frontend oder aus einer API – werden geprüft, bevor sie weiterverarbeitet oder gespeichert werden.
3. **Sinnvolle Datensparsamkeit:** Speichern Sie nur, was Sie wirklich brauchen. Wenn Daten lokal bleiben können, bleiben sie lokal.
4. **Dokumentation:** In Ihrer Anforderungsanalyse halten Sie fest, welche personenbezogenen Daten verarbeitet werden, wo sie gespeichert sind und wie sie geschützt sind.

Wer mehr will, kann zusätzlich umsetzen: Verschlüsselung der Datenbankdatei, Hashing eingegebener Passwörter (falls Sie eine Benutzerverwaltung bauen), Audit-Log für Datenzugriffe.

Was *nicht* verlangt wird

Damit Sie wissen, wo Sie sich nicht verkünsteln müssen:

- Eine umfangreiche Benutzerverwaltung ist nicht Pflicht.
- Es muss keine Cloud-Anbindung geben.
- Sie müssen keine Tests schreiben (dürfen aber gerne).
- Sie müssen die Anwendung nicht öffentlich hosten.
- Es muss nicht hübsch aussehen – es muss bedienbar und verständlich sein.

Wenn Sie mit dem Pflichtteil fertig sind, finden Sie in Abschnitt [Anregungen für die Kür](#).

Themenvarianten

Variante A – Strompreis- und Energieassistent

Idee: Strompreise an der Strombörse schwanken stündlich. Wer flexibel verbrauchen kann – Waschmaschine, Geschirrspüler, Wärmepumpe, E-Auto-Ladung – spart bares Geld, wenn er günstige Zeitfenster nutzt. Ihre Anwendung hilft dabei.

Was die Anwendung leisten soll:

1. Eine Nutzerin erfasst ihre üblichen Verbraucher mit Leistung und ungefährender Laufzeit (z. B. ?Waschmaschine, 2 kW, 1,5 h?).
2. Die Anwendung ruft die aktuellen oder kommenden Börsenstrompreise von einer Energie-API ab.
3. Sie berechnet die voraussichtlichen Kosten für jeden Verbraucher zu verschiedenen Zeitpunkten.
4. Sie zeigt eine Empfehlung an: ?Günstigster Zeitpunkt für Ihre Waschmaschine heute: 22:00 bis 23:30, geschätzte Kosten 0,28 Euro.?
5. Vergangene Empfehlungen und Preisverläufe werden gespeichert, damit Auswertungen möglich werden (z. B. durchschnittlicher Strompreis pro Wochentag).

Daten und Bibliotheken:

- **API:** aWATTar (<https://api.awattar.de/v1/marketdata>) liefert die deutschen Börsenstrompreise im Stundentakt ohne API-Schlüssel zurück. Alternative: **Energy-Charts** der Fraunhofer ISE.
- **Datenquelle:** SQLite-Datenbank mit den Tabellen `verbraucher`, `preise` (Historie) und `empfehlungen`.
- **pandas:** Aggregationen über Tag/Woche, Berechnung von Kosten, Identifikation günstigster Stunden.

Datenschutz-Aspekt: Verbrauchsverhalten ist personenbeziehbar. In Ihrer Anforderungsanalyse skizzieren Sie, wie die Daten geschützt sind – z. B. ausschließlich lokale Speicherung, kein Versand an Dritte, keine Verknüpfung mit Klarnamen, wenn nicht zwingend nötig.

Beispiel: Minimale Datenstruktur Variante A

Die Tabelle `verbraucher` könnte folgende Spalten haben:

- `id` – eindeutiger Schlüssel
- `bezeichnung` – z. B. ?Waschmaschine?
- `leistung_kw` – Stromaufnahme in Kilowatt
- `dauer_minuten` – typische Laufzeit
- `flexibel` – ob der Verbraucher zeitlich verschoben werden kann

Variante B – Reise- und Städte-Dashboard

Idee: Eine Anwendung, die selbst gewählte Reiseziele bündelt und vergleichbar macht. Wer überlegt, ob er im April nach Lissabon, Edinburgh oder Stockholm reisen will,

bekommt aktuelle Wetterdaten, Klimaprognosen, Reisedauer und Länderfakten in einer Ansicht.

Was die Anwendung leisten soll:

1. Die Nutzerin verwaltet eine Liste von ?Wunschstädten? (anlegen, ändern, löschen).
2. Pro Stadt werden aktuelle und prognostizierte Wetterdaten abgerufen.
3. Optional kommen Länderfakten hinzu: Hauptstadt, Sprache, Währung, Zeitzone.
4. Die Anwendung zeigt eine **Vergleichsansicht**: Welche Stadt hat in den nächsten Tagen die angenehmsten Temperaturen, die wenigsten Niederschläge, die meisten Sonnenstunden?
5. Die Wetterdaten der vergangenen Abrufe werden gespeichert, sodass Sie auch Verläufe und Statistiken anzeigen können.

Daten und Bibliotheken:

- **Wetter-API:** Open-Meteo (<https://open-meteo.com>) – kostenlos, ohne API-Schlüssel, sehr großzügig. Alternativ OpenWeatherMap (kostenloser API-Schlüssel auf Anfrage).
- **Optional Länder-API:** REST Countries (<https://restcountries.com>) – liefert Hauptstadt, Sprachen, Währungen.
- **Datenquelle:** SQLite mit Tabellen `staedte`, `wetter_historie`; alternativ Excel-Datei.
- **pandas:** Aggregation der Wetterdaten, Vergleichsberechnungen, Trendauswertungen.

Datenschutz-Aspekt: Die Daten sind weniger sensibel als bei Variante A, dennoch gelten dieselben Grundsätze: Keine API-Schlüssel im Code, validierte Eingaben, dokumentierte Datenflüsse.

Beispiel: Minimale Datenstruktur Variante B

Die Tabelle `staedte` könnte folgende Spalten haben:

- `id` – eindeutiger Schlüssel
- `name` – z. B. ?Lissabon?
- `land` – z. B. ?Portugal?
- `breitengrad`, `laengengrad` – Koordinaten für die Wetter-API
- `angelegt_am` – Zeitstempel

Welche Variante passt zu mir?

Beide Varianten sind gleichwertig und gleich anspruchsvoll. Eine kleine Entscheidungshilfe:

Variante A passt, wenn Sie...	Variante B passt, wenn Sie...	
sich für Energie, Markt und Optimierung interessieren	visuelle Vergleiche und Karten mögen	
gerne mit Zeitreihen arbeiten	gerne mit verschiedenen API-Endpunkten experimentieren	
ein klares, technisches Werkzeug bauen wollen	ein ?nutzbares Dashboard? bauen wollen	
Lust auf eine konkrete Empfehlungslogik haben	Lust auf Vergleichsdarstellungen haben	

Empfohlener Tech-Stack

Sie sind in der Wahl Ihrer Werkzeuge frei. Damit Sie aber nicht den ersten Tag mit Tool-Recherche verbringen, hier eine klare Empfehlung für den Einstieg und eine Liste anspruchsvollerer Optionen für die Kür.

Einstieg – der schnelle Weg zum laufenden Prototyp

Komponente	Empfehlung
------------	------------

Frontend Streamlit – ein Python-Befehl genügt für ein Eingabefeld, ein Diagramm, eine Tabelle.

Backend Eigene Python-Module – ganz schlicht, ohne zusätzliches Framework. Logik liegt in Modulen, Streamlit importiert sie.

Datenbank SQLite – über das Modul `sqlite3` aus der Python-Standardbibliothek. Eine Datei, kein Server.

HTTP / API requests – die Standard-Bibliothek für HTTP-Aufrufe in Python.

Datenanalyse pandas

Visualisierung Streamlit-Charts oder **plotly**

Tests (optional) pytest

Mit dieser Kombination ist ein erster lauffähiger Prototyp realistisch am Ende von Tag 2 möglich.

Kür – für die Ehrgeizigen

Wer mit dem Pflichtteil sicher durch ist, kann eine oder mehrere dieser Erweiterungen angehen:

- **Echtes Web-Backend mit Flask oder FastAPI:** Eigene Routen, eigene HTML-Templates (Jinja2), klare Trennung zwischen API-Schicht und Logik.
- **Benutzerverwaltung:** Registrierung, Login mit gehashten Passwörtern (`bcrypt` oder `argon2`), Session-Management.
- **Hosting:** Deployment der Anwendung über einen kostenlosen Hoster (z. B. Streamlit Community Cloud, Render, Fly.io). Sie haben dann eine echte URL, unter der Ihr Projekt erreichbar ist.
- **Interaktive Karten:** Bei Variante B mit `folium` oder `pydeck` eine Karte mit Ihren Wunschstädten.
- **Automatisierung:** Ein Hintergrundprozess (z. B. mit `schedule` oder `Cron`), der regelmäßig Daten holt und aktualisiert.
- **Erweiterte Auswertungen:** Trends erkennen, Anomalien melden, Prognosen rechnen.
- **Mehrere Ausgabeformate:** Export der Auswertungen als PDF, CSV, Excel.
- **Konfigurierbarkeit:** Alle Einstellungen in einer `.env`-Datei oder einer JSON-Konfiguration.
- **Tests:** Unit-Tests mit `pytest` – mindestens für die Kern-Geschäftslogik.
- **Logging:** Strukturiertes Logging mit dem `logging`-Modul statt `print()`.

Wichtig!

Kür ist Kür. Wer alle Pflichtanforderungen sauber umsetzt und nichts davon dazunimmt, bekommt genauso positives Feedback wie jemand mit Hosting und Benutzerverwaltung. Lieber Pflicht solide als Kür halb fertig.

Empfohlene Projektstruktur

Eine saubere Ordner- und Modulstruktur erspart Ihnen ab Tag 3 erfahrungsgemäß viele Stunden Such- und Fixarbeit. Hier ist ein Vorschlag, an dem Sie sich orientieren können:

Code-Beispiel: Empfohlene Projektstruktur

```
1 mein_projekt/  
2 +-- main.py           # Einstiegspunkt der Anwendung  
3 +-- gui/
```

```

4 |   +-- __init__.py
5 |   +-- app.py           # Streamlit- oder Flask-Code
6 +-- core/
7 |   +-- __init__.py
8 |   +-- api_client.py   # API-Aufrufe (requests)
9 |   +-- datenbank.py    # SQLite-Zugriff
10 |  +-- auswertung.py    # pandas-Auswertungen
11 |  +-- modelle.py       # Datenklassen / Schemas
12 +-- daten/
13 |  +-- meine_daten.db   # SQLite-Datenbank (nicht in Git!)
14 +-- konfiguration/
15 |  +-- einstellungen.py  # liest Werte aus .env
16 +-- tests/             # optional
17 |  +-- test_auswertung.py
18 +-- .env               # API-Schlüssel etc. (nicht in
    Git!)
19 +-- .gitignore
20 +-- requirements.txt
21 +-- README.md
22 +-- Anforderungsanalyse.md (oder .docx, .pdf)

```

Wozu die Trennung in gui/ und core/? Die Idee ist die schon angesprochene Schichtentrennung: gui/ kümmert sich um die Darstellung, core/ um die Sachlogik. Wenn Sie später Ihre Streamlit-Oberfläche gegen eine Flask-Oberfläche austauschen wollen, müssen Sie idealerweise nur in gui/ etwas ändern.

Praxistipp

Halten Sie main.py klein. Im Idealfall macht main.py nur drei Dinge: Konfiguration laden, Anwendung starten, eventuell eine erste Initialisierung anstoßen. Die eigentliche Logik gehört in die Module.

Empfohlene Entwicklungsreihenfolge

Die folgenden Meilensteine sind *Vorschläge*. Sie müssen sich nicht sklavisch daran halten. Aber wenn Sie an einem Tag deutlich hinter dem Plan sind, ist das ein Hinweis, dass Sie zurück zur Pflicht und weg von der Kür gehen sollten.

Tag 0 – Vorbereitungstag

Heute werden im Unterricht der Tech-Stack vorgestellt, Beispiele gezeigt und Fragen geklärt. Am Ende des Tages sollten Sie:

- Variante A oder Variante B gewählt haben,
- eine erste Idee für Ihre Projektstruktur skizziert haben,

- Python und Ihre IDE einsatzbereit haben,
- die wichtigsten Bibliotheken (`streamlit`, `requests`, `pandas`) installiert haben.

Tag 1 – Anforderungen und Skelett

Ziel des Tages: Sie wissen, was Sie bauen wollen, und haben das Projektgerüst angelegt.

1. Anforderungsanalyse erstellen (mindestens funktionale und nicht-funktionale Anforderungen, User Stories, Abgrenzung).
2. Ordnerstruktur anlegen, leere Modul-Dateien erstellen.
3. Virtuelle Umgebung anlegen, Bibliotheken installieren, `requirements.txt` pflegen.
4. `.gitignore` aufsetzen, falls Sie Git nutzen.
5. Erstes „Hallo Welt“ mit Streamlit oder Flask – nur damit der Stack läuft.

Tag 2 – API und Datenbank

Ziel: Sie können extern Daten abrufen und sie persistent speichern.

1. Ein kleines Test-Skript schreiben, das die API erfolgreich anspricht und die Antwort in der Konsole ausgibt.
2. Datenbankschema entwerfen und mit SQL anlegen.
3. Ein kleines Test-Skript, das in die Datenbank schreibt und wieder liest.
4. Beides sauber in Module verpacken (`api_client.py`, `datenbank.py`).

Tag 3 – Erster Durchstich

Ziel: Es gibt einen lauffähigen **Durchstich** – also einen vollständigen Pfad von der Oberfläche über die Logik bis zur Datenquelle und zurück.

1. Mindestens eine Funktion über die Oberfläche auslösbar, die API anspricht, in die Datenbank speichert und Daten zurück anzeigt.
2. Erste pandas-Auswertung in Ansätzen.
3. Erste, grobe Fehlerbehandlung an den kritischen Stellen.

Merksatz

Spätestens am Ende von Tag 3 sollte ein **laufender, wenn auch unvollständiger** Prototyp existieren. Funktioniert das nicht, bitte den Dozenten ansprechen – das ist genau der Punkt, an dem man frühzeitig steuern kann.

Tag 4 – Funktionsumfang ausbauen

Ziel: Alle Pflichtfunktionen sind in einer ersten Version vorhanden.

- Restliche Funktionen umsetzen.
- Datenverarbeitung mit `pandas` ausarbeiten.
- Oberfläche aufräumen, Bedienlogik klarer machen.

Tag 5 – Robustheit und Datenschutz

Ziel: Ihre Anwendung verträgt auch ungünstige Fälle.

- Fehlerbehandlung systematisch ergänzen (API nicht erreichbar, leere Eingaben, falsche Werte).
- Eingaben validieren.
- Konfiguration aus dem Code ziehen (.env-Datei).
- Datenschutz-Aspekte umsetzen und dokumentieren.
- Code aufräumen, Funktionen sinnvoll benennen, Docstrings ergänzen.

Tag 6 – Polishing oder Kür

Ziel: Die Pflicht ist solide. Heute ist Zeit für Kür oder für saubereres Polishing.

- Wer mit der Pflicht durch ist: ein oder zwei Kür-Themen aus Abschnitt angehen.
- Wer mit der Pflicht noch nicht durch ist: Pflicht abschließen. Keine neuen Baustellen!

Tag 7 – Doku, Test, Übergabe, Präsentation

Ziel: Die Abgabe steht und die Präsentation ist vorbereitet.

1. README.md fertigstellen.
2. Projektdokumentation (Designentscheidungen, Stärken, Schwächen, Potenziale) schreiben.
3. Anforderungsanalyse abschließend angleichen.
4. Anwendung an einem ?frischen Stand? testen – läuft sie nach `pip install -r requirements.txt` aus dem ZIP heraus?
5. ZIP-Archiv erstellen, abgeben.
6. Präsentation vorbereiten (Folien, Demo-Backup).

Umgang mit KI-Werkzeugen

KI-Tools wie ChatGPT, Claude, Copilot und ähnliche sind aus der täglichen Entwicklungsarbeit nicht mehr wegzudenken. Auch in Ihrer späteren Berufspraxis werden Sie damit arbeiten. Für dieses Projekt – als erste, eigene größere Anwendung – ist eine bewusste Auseinandersetzung mit diesen Werkzeugen wichtig.

Wichtig!

KI-Tools sind **erlaubt und erwünscht**, wenn sie Ihren Lernprozess unterstützen. Sie sind **problematisch**, wenn sie ihn ersetzen.

Sinnvoller Einsatz

- **Recherche und Verständnisaufbau:** ?Erkläre mir, was eine REST-API ist und wie sie sich von SOAP unterscheidet.?
- **Syntax-Hilfe:** ?Wie funktioniert groupby in pandas mit mehreren Spalten??
- **Fehlersuche:** ?Ich bekomme diesen Fehler – was bedeutet er und woran könnte das liegen??
- **Code-Review nach dem Schreiben:** ?Hier ist meine Funktion – siehst du Probleme oder Verbesserungen??
- **Alternativen vergleichen:** ?Welche Optionen gibt es, um ... zu lösen? Welche sind für meinen Fall sinnvoll??
- **Konzepte zusammenfassen:** ?Erkläre mir den Unterschied zwischen INNER JOIN und LEFT JOIN mit einem kleinen Beispiel.?

Schädlicher Einsatz

- ?Schreib mir das gesamte Projekt.?
- ?Hier ist die Aufgabe, gib mir die Lösung.?
- Code übernehmen, ohne ihn zu verstehen.
- Fehlermeldungen blind in den Chat kopieren und die erste Antwort ungeprüft übernehmen.
- KI nutzen, um sich Lernarbeit *zu ersparen* statt sie zu *erleichtern*.

Ein einfacher Test

Merksatz

Bevor Sie Code aus einer KI-Antwort übernehmen, müssen Sie diese drei Fragen mit ?Ja? beantworten können:

1. Verstehe ich, was dieser Code tut – Zeile für Zeile?
2. Könnte ich diesen Code in zwei Wochen einer Kollegin erklären?
3. Passt dieser Code zu dem, was ich vorhabe, oder ist er nur ?ungefähr richtig??

In Ihrer Präsentation am Ende werden Sie nach Designentscheidungen gefragt. Wer Code übernommen hat, ohne ihn zu verstehen, wird dort an Stellen scheitern, die mit etwas eigener Auseinandersetzung leicht zu beantworten gewesen wären.

Praxistipp

Notieren Sie sich Ihre KI-Nutzung. In der Projektdokumentation halten Sie kurz fest, wo KI Ihnen geholfen hat – nicht weil das „schlimm“ wäre, sondern weil es eine ehrliche Reflexion ist. Beispiel: „Den Aufbau der pandas-Pivot-Tabelle habe ich mir von ChatGPT erklären lassen und dann selbst implementiert.“

Optional Code-Review

Code-Review – also das gegenseitige Lesen und Besprechen von Code – ist eine der wirkungsvollsten Lernmethoden in der Softwareentwicklung. Sie ist in diesem Projekt nicht Pflicht, weil es Ihre erste eigene Anwendung ist und der Fokus auf der eigenen Erfahrung liegt.

Sie dürfen aber gerne paarweise Code-Reviews machen, wenn beide Beteiligten das wollen und die Zeit dafür da ist. Sinnvolle Punkte für Reviews sind etwa Ende Tag 3 (erster Durchstich) und Ende Tag 5 (Pflicht im Wesentlichen fertig).

Hinweis

Code-Review heißt nicht: *Lösungen abschauen*. Es heißt: *verstehen, wie jemand anders ein Problem gelöst hat*. Wer Code aus einem Review eins zu eins übernimmt, ohne ihn zu verstehen, hilft sich nicht. Diskutieren Sie stattdessen: Warum hast du es so gemacht? Was wäre eine Alternative? Was würdest du beim nächsten Mal anders machen?

Abgabe

Format

Wichtig!

Abgabe als ZIP-Archiv mit dem Namen
Nachname.Vorname.Projekt1.zip

Inhalt des ZIP-Archivs:

1. **Quellcode** – alle .py-Dateien in der entworfenen Projektstruktur,
2. **requirements.txt** – mit allen genutzten Bibliotheken und Versionen,
3. **README.md** – Beschreibung des Tools, Installation, Bedienung,
4. **Anforderungsanalyse** – Ihre überarbeitete Fassung,
5. **Projektdokumentation** – 1–2 Seiten mit Designentscheidungen, Stärken/Schwächen, Potenzialen,

6. **Beispiel-Output** – z. B. die SQLite-Datei mit Beispieldaten oder ein Screenshot der Anwendung,
7. **KEINE Geheimnisse** – keine `.env`, keine echten API-Schlüssel.

README.md – Mindeststruktur

Code-Beispiel: README.md Vorlage

```
1 # [Projektname]
2
3 ## Beschreibung
4 Kurzbeschreibung in zwei bis drei Sätzen.
5
6 ## Voraussetzungen
7 - Python 3.10 oder hoeher
8 - Bibliotheken siehe requirements.txt
9
10 ## Installation
11 1. Virtuelle Umgebung anlegen: python -m venv .venv
12 2. Aktivieren (Linux/Mac): source .venv/bin/activate
13   Aktivieren (Windows): .venv\Scripts\activate
14 3. Bibliotheken installieren: pip install -r requirements.txt
15 4. Konfiguration: .env.example nach .env kopieren und
16   ausfuellen
17
18 ## Bedienung
19 1. Anwendung starten: streamlit run main.py
20   (oder python main.py - je nach Variante)
21 2. ...
22
23 ## Hinweise zum Datenschutz
24 - Welche Daten werden lokal gespeichert?
25 - Wo liegt die Datenbank?
26 - Werden Daten an Dritte uebertragen?
27
28 ## Autor
29 Vorname Nachname, [Datum]
```

Präsentation

Format: Einzelpräsentation

Dauer: 8–10 Minuten pro Person, anschließend 3–5 Minuten Fragen

Termin: Tag 7 nach der Abgabe

Erwarteter Inhalt

1. **Kurzvorstellung des Projekts** (1 Minute) – Was tut Ihre Anwendung? Warum diese Variante? Welche Idee steckt dahinter?
2. **Live-Demo** (2–3 Minuten) – Sie zeigen Ihre Anwendung im Einsatz. Reichen Sie es nicht durch, sondern führen Sie typische Nutzungssituationen vor.
3. **Code-Walkthrough** (2–3 Minuten) – Eine oder zwei besonders interessante Stellen aus dem Code, am liebsten die, an denen Sie am meisten gelernt haben.
4. **Designentscheidungen begründen** (2 Minuten) – Warum SQLite und nicht Excel? Warum Streamlit und nicht Flask? Warum diese Modulgliederung? Sie müssen Ihre Wahl nicht verteidigen – nur *begründen*.
5. **Stärken, Schwächen, Potenziale** (1–2 Minuten) – Was gefällt Ihnen an Ihrem Projekt? Wo sehen Sie Schwachstellen? Was würden Sie als Nächstes ändern oder erweitern?

Praxistipp

Erstellen Sie ein Demo-Backup. Bereiten Sie Screenshots oder eine kurze Videoaufzeichnung vor. Wenn am Präsentationstag die API nicht erreichbar ist oder die WLAN-Verbindung ausfällt, sind Sie damit auf der sicheren Seite.

Was nicht gefragt ist

- Keine Selbstbewertungen mit Schulnoten.
- Keine Entschuldigungen für ?nur? Pflichtumfang.
- Keine wiederholte Erläuterung von Theorie – konzentrieren Sie sich auf *Ihre* Lösung.

Feedbackkriterien

Es gibt keine Note. Sie bekommen **konstruktives Feedback** zu folgenden Aspekten:

Definition

Pflichtumsetzung (40%)

- Sind alle sechs Pflichtanforderungen sichtbar umgesetzt?
- Läuft die Anwendung aus dem ZIP heraus stabil?
- Wurden Aspekte des Datenschutzes konkret berücksichtigt?

Code-Qualität und Struktur (25%)

- Ist der Code lesbar, sinnvoll benannt, sinnvoll modularisiert?
- Ist die Trennung Frontend / Backend / Datenquelle erkennbar?
- Sind Funktionen mit Docstrings dokumentiert?

Dokumentation und Anforderungsanalyse (15%)

- Passen Anforderungsanalyse und tatsächlich umgesetztes Projekt zusammen?
- Sind Designentscheidungen nachvollziehbar dokumentiert?
- Ist die README selbsterklärend?

Präsentation und Reflexion (20%)

- Können Sie Ihre Anwendung verständlich vorstellen?
- Können Sie Ihre Designentscheidungen begründen?
- Können Sie Stärken, Schwächen und Potenziale realistisch einschätzen?

Was nicht bewertet wird: Aussehen, Anzahl der Kür-Features, Vollständigkeit gegenüber dem theoretischen Maximum.

Häufige Fragen

F: Was, wenn ich mit dem Pflichtteil nicht fertig werde?

A: Konzentrieren Sie sich auf das Wesentliche. Lieber alle sechs Pflichtanforderungen *in einfacher Form* umgesetzt als zwei davon *perfekt* und die anderen vier offen.

F: Darf ich Bibliotheken nutzen, die hier nicht erwähnt sind?

A: Ja, gerne. Solange Sie sie verstehen, dokumentieren und in der `requirements.txt` auflisten.

F: Darf ich beide Varianten kombinieren?

A: Bitte nicht. Sie haben sieben Tage. Eine Variante sauber ist besser als zwei halb.

F: Was, wenn die API während meiner Präsentation nicht funktioniert?

A: Deswegen das Demo-Backup. Außerdem darf eine gute Anwendung damit umgehen – das ist Teil der Pflichtaufgabe ?Fehlerbehandlung?.

F: Darf ich mit anderen Teilnehmenden reden?

A: Reden ja, abschreiben nein. Austausch über Konzepte und Herangehensweisen ist erwünscht. Code-Sharing nicht.

F: Darf ich mein Projekt nach der Abgabe weiterentwickeln?

A: Unbedingt. Aber für die Bewertung zählt der Stand zur Abgabe.

F: Was, wenn ich keine Lust auf Streamlit habe und lieber Flask machen will?

A: Gerne. Sie sind frei in der Wahl. Achten Sie nur darauf, dass die Einarbeitung in Flask realistisch in Ihre Sieben-Tage-Planung passt.

F: Soll ich Git nutzen?

A: Empfehlung: ja. Auch wenn es nicht Pflicht ist. Versionskontrolle ist eine der wichtigsten Berufsfähigkeiten. Wenn Sie noch nie mit Git gearbeitet haben, ist dieses Projekt eine gute Gelegenheit für den Einstieg – aber nicht der Hauptlerngegenstand. Verzetteln Sie sich nicht.

Zum Schluss

Sie haben in den letzten Wochen viele kleine Bausteine gelernt – jetzt setzen Sie sie zum ersten Mal zu etwas *Eigenem* zusammen. Das ist anstrengend, manchmal frustrierend, und am Ende oft erstaunlich befriedigend. Lassen Sie sich nicht entmutigen, wenn am Tag 2 nichts funktioniert: Das geht erfahrenen Entwicklerinnen genauso. Was zählt, ist, dass Sie systematisch weiterarbeiten, sich Hilfe holen, wenn Sie nicht weiterkommen, und am Ende ein Programm haben, das Sie kennen, verstehen und erklären können.

Viel Erfolg!

Erik Schwob
Robotron Bildungszentrum Halle